



Time Series Forecasting Models in R: ARIMA, SVM, Random Forest, MLP, and LSTM

Time Series Forecasting Examples

Technical tutorial using the `tspredict` package in R. Each model family assumes distinct premises about the series — use ARIMA as a baseline, then progress through classical machine learning regressors, neural models, and finally automated hyperparameter search.

01

Classical Baseline

ARIMA with automatic order selection

02

ML Regressors

KNN, SVM and Random Forest

03

Neural Networks

MLP, ELM, Conv1D and LSTM

04

Automated Search

Model tuning and integrated tuning

ARIMA – Forecasting Baseline

```
# NOTE: ARIMA does not use a sliding window
(sw_size = 0)
# All other models follow the Standard
Workflow
library(tspredit)

# Load series (no sliding window)
data(tsd)
ts <- ts_data(tsd$y, 0)

# Split train/test and project (X, y)
samp <- ts_sample(ts, test_size = 5)
io_train <- ts_projection(samp$train)
io_test <- ts_projection(samp$test)

# Train ARIMA (automatic order selection)
model <- ts_arima()
model <- fit(model, x=io_train$input,
             y=io_train$output)
```

```
# Training adjustment
adjust <- predict(model, io_train$input)
ev_adjust <- evaluate(model,
  as.vector(io_train$output),
  as.vector(adjust))
# ev_adjust$mse => 0.02857686

# 5-step-ahead forecast
prediction <- predict(model,
  x=io_test$input[1,], steps_ahead=5)
ev_test <- evaluate(model,
  as.vector(io_test$output),
  as.vector(prediction))
# mse=0.490  smape=1.490  R2=-3.24
```

Step by step

- **ts_data(y, 0)**: loads the series without creating sliding windows
- **ts_sample**: preserves temporal order in the train/test split
- **ts_projection**: separates inputs (X) and targets (y)
- **ts_arima()**: automatically selects orders p, d, q
- **steps_ahead=5**: recursive 5-step forecast
- Training MSE **0.029** vs Test MSE **0.490** — sign of extrapolation difficulty

📌 **Key concept:** ARIMA models the univariate series using autoregressive terms, differencing, and moving averages. It serves as the protocol reference before any sliding window model.

DARIMA – Delegated ARIMA with Sliding Window

About the method

DARIMA (Delegated-differencing ARIMA) is a regressor inspired by ARIMA that operates within the sliding window workflow of `tspreedit` (lineage `ts_regsw`). Instead of embedding the integration order into the model, it delegates differencing to the preprocessing pipeline via `ts_norm_diff()`. This allows combining ARIMA logic with other components of the package. Main parameters: `input_size` and the preprocessor `ts_norm_diff()`. Reference: Box, Jenkins, Reinsel & Ljung (2015).

```
# Setup: see slide "Standard Workflow"
# DARIMA uses ts_norm_diff() instead of
# ts_norm_gminmax() — delegated differencing

# Train DARIMA (input_size=4)
model <- ts_darima(ts_norm_diff(),
                  input_size = 4)
model <- fit(model,
             x = io_train$input,
             y = io_train$output)

# Results
# train mse => (see example in repository)
# test mse  => 5 steps ahead
```

Step by step

- **`ts_darima()`**: ARIMA-like regressor in the `ts_regsw` lineage — compatible with the ML workflow
- **`ts_norm_diff()`**: takes on the role of differencing, making it explicit in the pipeline
- Different from classic ARIMA: operates on sliding windows, not on the entire series
- Allows combining differencing with other preprocessors in the package
- Useful as a univariate support block in target-centered multivariate forecasting

📌 **Key concept:** DARIMA brings ARIMA logic into the sliding window workflow, delegating integration to the preprocessor. Reference: Box et al. (2015).

WARMA – Window-based ARMA-Inspired Regressor

About the method

WARMA (Window-based ARMA) is a statistical regressor with local and stepwise normalization that operates within the `ts_regsw` lineage of `tspredict`. Unlike classical ARIMA, it applies per-window (local) normalization instead of global normalization, making it sensitive to local variations in the series. It can be combined with `ts_norm_diff()` in the external pipeline for delegated differencing, enabling a fairer comparison with ARIMA. Main parameters: `input_size` and `steps` (internal differencing steps). Reference: Ogasawara et al. (2010); WARMA manuscript SBBD 2026.

```
# Setup: see slide "Standard Workflow"
# WARMA uses ts_norm_diff() for delegated
# differencing (steps=0 disables internal one)

# Train WARMA (input_size=4, steps=0)
model <- ts_warma(ts_norm_diff(),
                  input_size = 4,
                  steps = 0)
model <- fit(model,
             x = io_train$input,
             y = io_train$output)

# Results
# mse train => (see example in repository)
# mse test  => 5 steps ahead
```

Step by step

- **ts_warma()**: ARMA-like regressor with local per-window (stepwise) normalization
- **steps=0**: disables internal differencing — delegated to external `ts_norm_diff()`
- Local normalization makes the model sensitive to level changes in the series
- Remains in the `ts_regsw` lineage: compatible with the standard ML workflow
- Useful as a local statistical competitor for univariate and multivariate forecasting

📌 **Key concept:** WARMA combines ARMA logic with local per-window normalization, making it a natural statistical competitor within the sliding window pipeline. Reference: Ogasawara et al. (2010).

Standard Workflow – Forecasting Pipeline

```
library(daltoolbox)
library(tspredit)

# 1. Load and window the series
data(tsd)
ts <- ts_data(tsd$y, 10)

# 2. Split train / test
samp <- ts_sample(ts, test_size = 5)

# 3. Project into (X, y)
io_train <- ts_projection(samp$train)
io_test <- ts_projection(samp$test)

# 4. Instantiate and train the model
# <- detailed in each following slide
model <- ts_<method>(ts_norm_gminmax(),
  input_size = ...)
model <- fit(model,
  x = io_train$input,
  y = io_train$output)
```

```
# 5. Evaluate fit on training set
adjust <- predict(model, io_train$input)
ev_adjust <- evaluate(model,
  as.vector(io_train$output),
  as.vector(adjust))
# -> mse, smape, R2

# 6. Forecast steps ahead on test set
prediction <- predict(model,
  x = io_test$input[1,],
  steps_ahead = 5)
ev_test <- evaluate(model,
  as.vector(io_test$output),
  as.vector(prediction))
# -> mse, smape, R2
```

Step by step

- **ts_data(y, sw)**: windows the series into lag vectors of size `sw`
- **ts_sample()**: splits preserving temporal order (no shuffling)
- **ts_projection()**: separates inputs `X` and targets `y` for each partition
- **ts_norm_gminmax()**: global min-max normalization embedded in the pipeline
- **ts_<method>()**: instantiates the learner — specific parameters in each slide
- **fit() / predict() / evaluate()**: unified interface for all models

📌 **Key concept:** This pipeline is reused across all examples. Only the `ts_<method>(...)` block changes between slides — the rest remains identical.

KNN – Regression by Nearest Neighbors

About the method

KNN (K-Nearest Neighbors) for regression searches the historical series for the k lag windows most similar to the current window and predicts the next value as a weighted average of the corresponding targets. It assumes no functional form — it is a non-parametric distance-based method. Main parameters: `input_size` (number of lags used as input vector) and k (number of neighbors consulted). Reference: Cover & Hart (1967).

```
# Setup: see slide "Standard Workflow"

# Train KNN (input_size=4, k=3)
model <- ts_knn(ts_norm_gminmax(),
               input_size=4, k=3)
model <- fit(model,
             x=io_train$input,
             y=io_train$output)

# Results
# mse train => 0.001692
# mse test=0.0373 smape=0.889 R2=0.678
```

Step by step

- **ts_data(y, 10)**: creates sliding windows of size 10 ($t_9 \dots t_0$)
- **ts_norm_gminmax()**: global min-max normalization embedded in the model
- **input_size=4**: uses the 4 most recent lags as input
- **k=3**: searches the 3 nearest neighbors to predict
- Test MSE **0.037** and R^2 **0.678** — significant improvement over ARIMA

📌 **Key concept:** KNN searches past windows similar to the current one and predicts from their targets. It is a non-parametric local baseline useful for evaluating distance-based behavior.

SVM – Support Vector Regression

About the method

SVR (Support Vector Regression) learns a non-linear function that keeps the prediction error within an epsilon margin, penalizing only the points outside it. Strong regularization (parameter C) controls the trade-off between fitting and generalization. Suitable when the relationship between lags and target is non-linear but smooth. Main parameters: `input_size`, `kernel`, `epsilon` and `cost`. Reference: Smola & Scholkopf (2004).

```
# Setup: see slide "Standard Workflow"

# Train SVM (input_size=4)
model <- ts_svm(ts_norm_gminmax(),
               input_size=4)
model <- fit(model,
             x=io_train$input,
             y=io_train$output)

# Results
# mse train => 5.13e-07
# mse test=3.2e-07  smape=0.00268
R2=0.9999972
```

Step by step

- **ts_svm()**: learns a non-linear prediction function by maximizing the margin
- **input_size=4**: defines the lag vector used as input
- Train MSE **5.1×10^{-7}** — near-perfect fit
- Test MSE **3.2×10^{-7}** and $R^2 \approx 1.000$ — excellent generalization in this example
- Useful when local linearity is insufficient but strong regularization helps

📌 **Key concept:** SVR learns a non-linear function by controlling the prediction error within a margin. Reference: Smola & Schölkopf (2004).

Random Forest – Ensemble of Trees

About the method

Random Forest builds an ensemble of decision trees, each trained on a bootstrap sample and using a random subset of features (lags). The final prediction is the average of the individual outputs. It captures non-linear interactions between lags without parametric assumptions and is robust to overfitting. Main parameters: `input_size`, `ntree` (number of trees) and `nodesize` (minimum leaf size). Reference: Breiman (2001).

```
# Setup: see slide "Standard Workflow"

# Train Random Forest
# input_size=8, nodesize=1, ntree=20
model <- ts_rf(ts_norm_gminmax(),
               input_size=8,
               nodesize=1,
               ntree=20)
model <- fit(model,
             x=io_train$input,
             y=io_train$output)

# Results
# mse train => 0.008343
# mse test=0.0653  smape=1.146  R2=0.436
```

Step by step

- **ntree=20**: number of trees in the ensemble
- **nodesize=1**: allows leaves with a single observation (low bias)
- **input_size=8**: window of 8 lags as features
- Captures non-linear interactions between lags without parametric assumptions
- Test R^2 **0.436** — moderate performance; strong ensemble with little feature engineering

📄 **Key concept:** Random Forests combine many trees trained on bootstrap samples and random subsets of features. Reference: Breiman (2001).

MLP – Multilayer Perceptron

About the method

The MLP (Multilayer Perceptron) is a feedforward neural network with one or more hidden layers of non-linear neurons. Trained via backpropagation, it maps the lag vector to the future value through successive non-linear transformations. The `decay` parameter controls L2 regularization to prevent overfitting. Main parameters: `input_size`, `size` (neurons in the hidden layer) and `decay`.

Reference: Rumelhart, Hinton & Williams (1986).

```
# Setup: see slide "Standard Workflow"

# Train MLP
# input_size=4, size=4 neurons, decay=0
model <- ts_mlp(ts_norm_gminmax(),
               input_size=4,
               size=4,
               decay=0)
model <- fit(model,
            x=io_train$input,
            y=io_train$output)

# Results
# train mse => 1.286e-05
# test mse=3.79e-06 smape=0.00863
R2=0.9999673
```

Step by step

- **size=4**: 4 neurons in the hidden layer
- **decay=0**: no L2 regularization (suitable for simple series)
- Trained via backpropagation on normalized lags
- Test MSE **3.8×10^{-6}** and $R^2 \approx 1.000$ — excellent generalization
- Central pipeline reused in the following neural examples

❏ **Key concept:** MLP maps input lags to the forecast target through non-linear hidden layers. Reference: Rumelhart, Hinton & Williams (1986).

Torch MLP – MLP via PyTorch Backend

About the method

The `torch_ts_mlp()` is a feedforward neural network implemented via PyTorch backend through the `daltoolboxdp` package. It maintains the same sliding window protocol as other regressors, but replaces the classic `nnet` learner with a deeper and more configurable MLP. Allows defining multiple hidden layers with `hidden_sizes`, controlling `epochs` and `batch_size`. Requires the `daltoolboxdp` package. Reference: Bishop (1995); Paszke et al. (2019).

```
library(daltoolbox)
library(daltoolboxdp) # additional package
library(tspredit)

# Setup: see slide "Standard Workflow"

# Train Torch MLP
# hidden_sizes=c(32L, 16L), epochs=200
model <- torch_ts_mlp(
  ts_norm_gminmax(),
  input_size = 4,
  hidden_sizes = c(32L, 16L),
  epochs = 200L,
  batch_size = 8L
)
model <- fit(model,
  x = io_train$input,
  y = io_train$output)

# Results
# mse train => (see example in repository)
# mse test => 5 steps ahead
```

Step by step

- **torch_ts_mlp()**: feedforward MLP via PyTorch — deeper and more configurable than `ts_mlp()`
- **hidden_sizes=c(32L, 16L)**: defines two hidden layers with 32 and 16 neurons
- **batch_size=8L**: training in mini-batches — more stable than full gradient
- **epochs=200L**: number of training epochs (fewer than Conv1D/LSTM since it is feedforward)
- Requires `daltoolboxdp`; compare with `ts_mlp()` to evaluate the gain from the PyTorch backend

❏ **Key concept:** `torch_ts_mlp()` is the PyTorch alternative to the classic `ts_mlp()` — same interface, deeper and more configurable learner. Reference: Paszke et al. (2019).

ELM – Extreme Learning Machine

About the method

ELM (Extreme Learning Machine) is a single-hidden-layer neural network whose hidden layer weights are randomly initialized and fixed — only the output layer weights are adjusted via least squares. This makes training extremely fast compared to the iterative backpropagation of MLP. Main parameters: `input_size`, `nhid` (hidden neurons) and `actfun` (activation function). Reference: Huang, Zhu & Siew (2006).

```
# Setup: see slide "Standard Workflow"

# Train ELM
# input_size=4, nhid=3, actfun="purelin"
model <- ts_elm(ts_norm_gminmax(),
               input_size=4,
               nhid=3,
               actfun="purelin")
model <- fit(model,
            x=io_train$input,
            y=io_train$output)

# Results
# mse train => 2.445e-27
# mse test=3.68e-25  smape~0  R2=1
```

Step by step

- **nhid=3**: 3 hidden neurons with fixed random weights
- **actfun="purelin"**: linear activation function in the hidden layer
- Only the output layer is adjusted (least squares)
- Training MSE **2.4×10^{-27}** and test $R^2 = 1$ — perfect fit in this example
- Training is much faster than MLP with iterative backpropagation

📌 **Key concept:** ELM uses random hidden weights and only adjusts the output. Much faster than backpropagation. Reference: Huang, Zhu & Siew (2006).

Conv1D – 1D Convolutional Network

About the method

Conv1D applies learnable convolutional filters over the lag window to detect local temporal patterns (motifs). Each filter slides across the input sequence and produces an activation map that captures short-range dependencies. Requires the `daltoolboxdp` package (PyTorch backend). Main parameters: `input_size` and `epochs`. Useful when short, repeated patterns dominate the series. Reference: LeCun et al. (1998).

```
library(daltoolbox)
library(daltoolboxdp) # additional package
library(tspredit)

# Setup: see slide "Standard Workflow"

# Train Conv1D
# input_size=4, epochs=10000
model <- ts_conv1d(ts_norm_gminmax(),
  input_size=4,
  epochs=10000)
model <- fit(model,
  x=io_train$input,
  y=io_train$output)

# Results
# train mse => 5.062e-06
# test mse=1.15e-04 smape=0.046 R²=0.999
```

Step by step

- Requires additional package `daltoolboxdp`
- **epochs=10000**: long training for adequate convergence
- Convolutional filters detect local temporal patterns in the window
- Test MSE **1.15×10^{-4}** and R^2 **0.999** — excellent performance
- Useful when short, repeated patterns matter more than global dependencies

📌 **Key concept:** Conv1D applies learnable filters over the lag window to detect local temporal motifs. Reference: LeCun et al. (1998).

LSTM – Long Short-Term Memory

About the method

LSTM (Long Short-Term Memory) is a recurrent network with gates (input, forget, output) that control the flow of information along the sequence. Unlike MLP and Conv1D, LSTM maintains an internal state that persists between steps, making it ideal for capturing long-range dependencies. Requires the `daltoolboxdp` package. Main parameters: `input_size` and `epochs`. Reference: Hochreiter & Schmidhuber (1997).

```
library(daltoolbox)
library(daltoolboxdp) # additional package
library(tspredit)

# Setup: see slide "Standard Workflow"

# Train LSTM
# input_size=4, epochs=10000
model <- ts_lstm(ts_norm_gminmax(),
  input_size=4,
  epochs=10000)

model <- fit(model,
  x=io_train$input,
  y=io_train$output)

# Results (steps_ahead=1)
# mse train => 0.0002032
# mse test=1.51e-04 smape=7.16% R2=0.9987
```

Step by step

- LSTM uses gates (input, forget, output) to retain long-range dependencies
- **steps_ahead=1**: step-by-step forecast over the test set
- Output formatted with `sprintf("%.2f, %.2f", output, prediction)`
- Test MSE **1.51×10^{-4}** , smape **7.16%**, R² **0.999**
- Compare with MLP and Conv1D to evaluate the gain from sequential dependencies

📌 **Key concept:** LSTM retains and updates information across ordered inputs. Ideal when sequential dependencies span more than just a few lags. Reference: Hochreiter & Schmidhuber (1997).

Model Tuning – Hyperparameter Search

About the method

The `ts_tune()` function performs hyperparameter search over a base model within the `tspremit` pipeline. The search simultaneously covers the window size (`input_size`) and the internal hyperparameters of the learner (e.g.: `nhid`, `actfun` for ELM; `size`, `decay` for MLP; `nodesize`, `ntree` for RF). The best model is returned already fitted on the training data, with no need to retrain manually. Uses a cosine series as an example (different from previous slides). Reference: Bergstra & Bengio (2012).

```
# Setup analogous to Standard Workflow
# (cosine series, sw_size=10, test_size=1)

# Define hyperparameter search
tune <- ts_tune(
  input_size = c(3:5),
  base_model = ts_elm(ts_norm_gminmax()),
  ranges = list(
    nhid = 1:5,
    actfun = c('sig','radbas','tribas',
               'relu','purelin')
  )
)

# Fit the best model
model <- fit(tune,
  x=io_train$input,
  y=io_train$output)

# Results
# Train: mse~3.2e-30 R2=1
# Test: output=0.99 prediction=0.99
```

Step by step

- **ts_tune()**: searches over the base model's hyperparameters without changing the rest of the pipeline
- **input_size=c(3:5)**: tests windows of 3, 4 and 5 lags
- Available ranges by model: ELM (`nhid`, `actfun`), MLP (`size`, `decay`), RF (`nodesize`, `ntree`), SVM (`kernel`, `epsilon`, `cost`), LSTM/CNN (`input_size`, `epochs`)
- The best model is returned already fitted on the training data

📌 **Key concept:** Performance depends on both the choice of learner and the representation (window size). `ts_tune` automates this search. Reference: Bergstra & Bengio (2012).

Integrated Tuning – Preprocessing + Model

About the method

The `ts_integtune()` extends `ts_tune()` by simultaneously optimizing preprocessing, window size, and learner hyperparameters in a single call. The `preprocess` parameter accepts a list of candidate transformations that are evaluated alongside the other hyperparameters. Evaluation respects temporal order (no look-ahead bias). Returns a fully configured complete pipeline with the best combination found. Reference: Salles et al. (2023) — TSPredIT, LNCS.

```
# Setup analogous to the Standard Workflow
# (cosine series, sw_size=10, test_size=1)

# Integrated tuning: preprocessing + model
tune <- ts_integtune(
  input_size = c(3:5),
  base_model = ts_elm(),
  preprocess = list(ts_norm_gminmax()),
  ranges = list(
    nhid = 1:5,
    actfun = c('sig','radbas','tribas',
               'relu','purelin')
  )
)

# Fit complete pipeline
model <- fit(tune,
  x=io_train$input,
  y=io_train$output)

# Results
# Train: mse~3.2e-30 R2=1
# Test: output=0.99 prediction=0.99
```

Step by step

- **ts_integtune()**: optimizes preprocessing, window size, and model hyperparameters jointly
- **preprocess=list(...)**: list of candidate transformations evaluated during the search
- Evaluation uses resampling that respects temporal order (no look-ahead bias)
- Returns a fully configured complete pipeline with the best combination found
- Reference: Salles et al. (2023) — TSPredIT, LNCS

📌 **Key concept:** Integrated tuning simplifies model selection by returning a fitted pipeline with the best configuration, covering preprocessing, windowing, and learner in a single call.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

